# Memoria Práctica 1 IA

Alejandro González García y Arturo Pérez Noves

Grupo 1311 | pareja 08

1. Respuestas de la Parte A

## Pregunta 1.1: ¿Cómo evoluciona el número de nodos al aumentar la profundidad?

El número de nodos aumenta rápidamente con la profundidad. Si el factor de ramificación medio es aproximadamente b, entonces el número de nodos crece aproximadamente como: 1+b+b2+···+b^d Por eso pasar de profundidad 3 a 4 puede aumentar mucho el coste.

## Pregunta 1.2: ¿Es viable explorar exhaustivamente el árbol completo de Reversi hasta el final de la partida? Justifica tu respuesta.

No es viable explorar exhaustivamente toda una partida de Reversi. En cada turno suele haber varias jugadas legales y una partida puede durar muchas jugadas. El crecimiento aproximado es exponencial: O(b^d) Aunque una posición tenga solo unas pocas jugadas posibles, al multiplicarlas durante decenas de turnos aparecen cantidades enormes de estados. Por eso después se utilizan profundidad limitada, heurísticas y poda alfa-beta.

## Pregunta 1.3: ¿Qué diferencias observáis entre BP y BA al explorar árboles de juego?

BP explora una rama hasta el fondo antes de volver atrás para explorar otra. Por eso puede encontrar una solución rápidamente, pero la solución no tiene por qué ser la más corta. En nuestro experimento BP encontró una solución de 9 movimientos visitando 10 nodos.BA explora primero todos los nodos de una profundidad, después los de la siguiente, y así sucesivamente. Por eso encuentra la solución con menos movimientos cuando todas las acciones tienen el mismo coste. En el experimento BA encontró una solución de 4 movimientos, aunque tuvo que visitar 80 nodos.

## Pregunta 1.4: ¿Cuál de los dos algoritmos requiere más memoria? Explica por qué.

BA requiere normalmente más memoria. Debe conservar en la frontera todos los nodos pendientes de un nivel antes de pasar al siguiente. Si el factor de ramificación es b y la solución está a profundidad d, puede llegar a almacenar O(b^d) nodos. BP necesita conservar principalmente la rama actual y las alternativas pendientes. Su memoria es aproximadamente O(bd) con una implementación de búsqueda en árbol, por lo que suele consumir menos memoria, aunque puede tardar más o encontrar una solución peor.

## Pregunta 1.5: Suponed que cada hoja contiene una puntuación fija (por ejemplo, en función del número de fichas negras). ¿Permiten estos algoritmos determinar cuál es la mejor jugada para un jugador racional? Justificad vuestra respuesta.

No, BP y BA por sí solos no determinan la mejor jugada. Estos algoritmos buscan un objetivo y devuelven la primera solución que encuentran; no comparan las puntuaciones de todas las hojas ni tienen en cuenta que el adversario también elige sus movimientos. Para elegir la mejor jugada habría que explorar los estados hasta una profundidad determinada, evaluar sus hojas y aplicar minimax: maximizar la puntuación en los turnos propios y minimizarla en los turnos del adversario. La poda alfa-beta puede reducir el número de nodos explorados sin cambiar la decisión.

## Pregunta 1.6: Define 3 funciones heurísticas con varios niveles de complejidad (puedes empezar a partir de la heurística entregada simpleHeuristic), implementando las funciones ya definidas heuristic1, heuristic2, heuristic3 (de menor a mayor complejidad) ¿Qué características parecen ayudar a resolver el problema de capturar una esquina en un menor número de pasos?

Hemos definido las tres heurísticas. heuristic1 es la heurística nula y no aporta información adicional. heuristic2 devuelve 0 si ya se ha capturado una esquina y 1 en caso contrario, que es una cota inferior del número de movimientos restantes. heuristic3 cuenta las esquinas que todavía no están capturadas. Para el problema de una esquina, las dos últimas heurísticas son equivalentes; para el problema de todas las esquinas, heuristic3 aporta más información.

En el experimento de una esquina, A* con heuristic1 visitó 80 nodos y encontró un camino de 4 movimientos. Con heuristic2 y heuristic3 visitó 72 nodos y también encontró un camino de 4 movimientos. Por tanto, conocer el objetivo y valorar el control de las esquinas ayuda a reducir la exploración sin cambiar la solución óptima.

## Pregunta 1.7: Analiza qué ocurre con los métodos de búsqueda y las distintas heurísticas propuestas si se intenta resolver el problema de capturar todas las esquinas, es decir, en el main, instancia el problema AllCornersReversiSearchProblem en lugar de CornerReversiSearchProblem.

Al usar AllCornersReversiSearchProblem, el objetivo exige capturar las cuatro esquinas en lugar de solo una. El espacio de estados crece mucho porque cada movimiento genera nuevas respuestas del adversario y hay que mantener la búsqueda durante muchas más jugadas. En el tablero de 6x6 utilizado, las búsquedas A* no terminaron en el tiempo de prueba de 8 segundos con ninguna de las tres heurísticas. Esto muestra que una heurística sencilla basada únicamente en el número de esquinas capturadas no guía lo suficiente la búsqueda para este objetivo más difícil.

## Pregunta 1.8: ¿Puede equivocarse la función heurística? ¿Qué aspectos habría que tener en cuenta para que A* sea óptimo?

Sí, una heurística puede valorar como prometedor un estado que en realidad está lejos del objetivo, o incluso sobreestimar el coste restante, nos ha pasado en las heurísticas del torneo. Para que A* sea óptimo, la heurística debe ser admisible, nunca debe estimar un coste mayor que el coste real mínimo hasta una solución. También conviene que sea consistente, de modo que el valor no disminuya más que el coste de una acción entre dos estados consecutivos. Las heurísticas utilizadas aquí son cotas inferiores: cada movimiento puede capturar como máximo una esquina, por lo que el número de esquinas que faltan no sobreestima los movimientos necesarios. En cambio, una heurística que premie muchas fichas, movilidad o posiciones estratégicas puede ser útil para guiar la búsqueda, pero no garantiza por sí sola que A* encuentre el camino óptimo si sus valores no son cotas inferiores del coste restante.

2. Minimax y poda alfa-beta

**Explicación vista en teoría:**

El jugador minimax construye sucesores hasta una profundidad limitada. En los nodos MAX selecciona el mayor valor y en los nodos MIN selecciona el menor. La evaluación de las hojas se expresa desde la perspectiva del jugador MAX. La poda alfa-beta mantiene dos cotas: alfa es el mejor valor garantizado para MAX y beta el mejor valor garantizado para MIN; cuando alfa >= beta, el resto de la rama no puede modificar la decisión y se omite.

## 2.1 Diseño e implementación

1. Bases de la implementación:

La implementación se basa en una jerarquía de estrategias. Tanto MinimaxStrategy como MinimaxAlphaBetaStrategy heredan de la clase abstracta Strategy que pertenece al material de la práctica.

La clase base proporciona:

- El atributo verbose, utilizado para mostrar información de depuración.

- El método next_move(), que debe ser implementado por cada estrategia.

- El método generate_successors, que obtiene los estados sucesores mediante el objeto TwoPlayerGame

Las estrategias no manipulan directamente la representación concreta del tablero. En su lugar, trabajan con objetos TwoPlayerGameState, que contienen:

- El tablero actual.

- El jugador que debe mover.

- El jugador considerado como MAX.

- El movimiento que produjo el estado.

- El estado padre.

- La información de final de partida y puntuación.

Esta separación permite utilizar las mismas estrategias con diferentes juegos, como el árbol simple, tres en raya o Reversi.

2. Explicacion de las estructuras de datos usadas:

La implementación utiliza las siguientes estructuras principales:

-Estados del juego

Cada nodo del árbol se representa mediante un objeto TwoPlayerGameState.

Esto permite almacenar no solo el tablero, sino también la relación con los estados anteriores y los datos necesarios para decidir si el estado es terminal.

En el caso del árbol simple, el tablero se representa mediante una cadena:

'A', 'B', 'C', ..., 'P'

El diccionario successor_lists de SimpleGameTree define las conexiones entre nodos:

```
{
    'A': ['B', 'C', 'D'],
    'B': ['E', 'F'],
    'C': ['G', 'H'],
    'D': ['I', 'J'],
    'G': ['K', 'L'],
    'J': ['M', 'N', 'O', 'P']
}
```

Las puntuaciones terminales se almacenan en otro diccionario:

```
{
    'E': [4, 0],
    'F': [3, 0],
    'H': [5, 0],
    'I': [5, 0],
    'K': [2, 0],
    'L': [1, 0],
    'M': [4, 0],
    'N': [2, 0],
    'O': [6, 0],
    'P': [1, 0]
}
```

- Lista de sucesores

Los sucesores de un estado se devuelven mediante una lista. El orden de esta lista es importante para la poda alfa-beta, ya que un buen orden de exploración puede producir más podas.

En este ejercicio se mantiene el orden definido en SimpleGameTree.

-Heurística

La heurística se encapsula en la clase Heuristic. Esta clase recibe una función que transforma un estado en un número real:

```
Heuristic(
    name='...',
    evaluation_function=...
)
```

Los valores se interpretan desde el punto de vista del jugador MAX:

- Valores positivos: ventaja para MAX.

- Valores negativos: ventaja para MIN.

- Valores altos: estados más favorables para MAX.

Para validar el árbol de la imagen se utilizó una heurística determinista que devuelve directamente el valor de las hojas.

3. Explicación del funcionamiento de las clases:

En la clase MinimaxStrategy implementamos el algoritmo Minmax con una profundidad máxima dada. La idea es representar dos tipos de decisiones:

- MAX: nuestra heurística intenta obtener la mayor puntuación posible.

- MIN: el oponente intenta reducir la puntuación de MAX.

Flujo principal

En next_move():

```
minimax_value, minimax_successor = self._max_value(
    state,
    self.max_depth_minimax,
)
```

La búsqueda comienza llamando a max_value(), porque se considera que el jugador que toma la decisión en la raíz es MAX.

El método max_value()

1. Comprueba si se ha alcanzado:

   - El final de la partida.

   - La profundidad máxima.

2. Si se ha alcanzado, evalúa el estado mediante la heurística.

3. Si no, genera todos los sucesores posibles.

4. Explora cada sucesor llamando a min_value()

5. Se queda con el sucesor que tenga el valor más alto.

Y el método min_value() funciona de forma equivalente, pero al contrario:

1. Evalúa directamente si es un estado terminal o se ha llegado al límite de profundidad.

2. Si no, genera los sucesores.

3. Explora cada uno llamando a max_value()

4. Elige el valor más pequeño, ya que MIN representa al oponente.

La alternancia entre max_value() y min_value() simula los turnos de ambos jugadores:

MAX → MIN → MAX → MIN → ...

En la clase MinimaxAlphaBetaStrategy implementamos el algoritmo Minmax con la poda alpha beta. El resultado final debería ser el mismo que con Minimax normal. La diferencia es que se evitan explorar ramas que ya no pueden cambiar la decisión final.

La búsqueda comienza con:

alpha = -np.inf

beta = np.inf

Esto significa que inicialmente no conocemos ninguna cota útil.

Alpha representa la mejor puntuación que MAX puede garantizar hasta ese momento.

Se actualiza en los nodos MAX:

alpha = max(alpha, minimax_value)

Beta representa la mejor puntuación que MIN puede garantizar hasta ese momento.

Se actualiza en los nodos MIN:

beta = min(beta, minimax_value)

Cuándo se realiza la poda

Poda en un nodo MIN

En min_value() aparece:

```
if minimax_value <= alpha:
    break
```

Esto significa:

- MAX ya tiene una alternativa con valor alpha o superior.

- MIN está encontrando una opción con valor menor o igual que alpha.

- Por tanto, MAX nunca escogería llegar a esta rama.

- Se pueden ignorar los sucesores restantes.

Poda en un nodo MAX

En max_value() aparece:

if minimax_value >= beta:

  break

Esto significa justo lo inverso a lo anterior:

- MIN ya tiene una alternativa con valor `beta` o inferior.

- MAX está encontrando una opción con valor mayor o igual que `beta`.

- MIN nunca permitiría llegar a esta rama.

- Se pueden ignorar los sucesores restantes.

Evaluación de las hojas

Ambas estrategias evalúan un estado cuando:

state.end_of_game or depth == 0

Es decir:

- La partida ha terminado.

- Se ha alcanzado la profundidad máxima de búsqueda.

En ese momento se ejecuta:

self.heuristic.evaluate(state)

La heurística es la que decide qué tan bueno es el tablero para MAX.

Por ejemplo:

- En tres en raya, podría valorar una victoria como +1 y una derrota como -1

- En Reversi, podría valorar la diferencia de fichas, las esquinas ocupadas o la movilidad.

Control del tiempo de la heurística con la librería time

Las dos clases también miden cuánto tarda la heurística:

time0 = time.time()

minimax_value = self.heuristic.evaluate(state)

time1 = time.time()

Si la evaluación tarda más que max_sec_per_evaluation se activa:

self.timed_out = True

A partir de entonces, las evaluaciones posteriores devuelven 0. Esto evita que una heurística demasiado lenta bloquee la partida.

4. Tests diseñados y aplicados

Hemos diseñado pruebas de corrección y de eficiencia. Y hemos realizado las pruebas pedidas en el enunciado.

Test 1: evaluación manual del árbol

Hicimos manualmente el árbol comenzando en la raíz A.

Los valores de los nodos son:

| Nodo | Cálculo | Valor |

| B | min(4, 3) | 3 |

| G | max(2, 1) | 2 |

| C | min(2, 5) | 2 |

| J | max(4, 2, 6, 1) | 6 |

| D | min(5, 6) | 5 |

| A | max(3, 2, 5)| 5 |

Por tanto, el valor minimax de la raíz es 5

El movimiento óptimo es:

A → D

Después, si se continúa la partida con un jugador MIN en el nodo D, este elige

D → I porque I tiene valor 5 y J tiene valor 6, y MIN selecciona el menor valor.

La secuencia esperada de la partida es:

A → D → I

Test 2: Comprobar que la ejecución coincide con la de demo_simple_game_tree.py

Resultado:

It is the turn of player 'Minimax 1' [Player 1].

A: -inf

B: inf

E: 4

B: 4

F: 3

B: 3

A: 3

C: inf

G: -inf

K: 2

G: 2

L: 1

G: 2

C: 2

H: 5

C: 2

A: 3

D: inf

I: 5

D: 5

J: -inf

M: 4

J: 4

N: 2

J: 4

O: 6

J: 6

P: 1

J: 6

D: 5

A: 5

Game state before move:

A

Minimax value = 5

Player 'Minimax 1' [Player 1] moves AD.

El resultado es exactamente el mismo que el manual.

Test 3: Poda alfa beta manual

Comenzando en A, con MAX como jugador inicial. La búsqueda empezó con:

Nota infty = infinito.

[alpha,beta]=[-infty,+infty]

Los valores obtenidos fueron:

B=min(4,3)=3

G=max(2,1)=2

C=min(2,5)=2

J=max(4,2,6,1)=6

D=min(5,6)=5

Finalmente, en la raíz:

A=\max(3,2,5)=5

Por tanto, el valor minimax de la raíz es 5

y MAX elige el movimiento:

A → D

Durante la búsqueda se producen dos podas:

- Se poda H al evaluar C, porque se cumple:

beta=2 <= alpha=3

- Se poda P al evaluar J, porque se cumple:

alpha=6 >=  beta=5

Los intervalos principales observados fueron:

A: [-∞, ∞]

B: [-∞, ∞]

A: [3, ∞]

C: [3, ∞]

D: [3, ∞]

D: [3, 5]

J: [3, 5]

J: [4, 5]

A: [5, ∞]

Test 4: comprobar resultado con demo_simple_game_tree.py

La salida con `verbose=3` de `demo_simple_game_tree.py` coincide con el cálculo manual:

Minimax value = 5

Player 'Minimax + alpha-beta 1' moves AD.

Después, el segundo jugador responde:

D → I

La secuencia completa de la partida es:

A → D → I

La comprobación confirma que:

| Elemento | Resultado |

| Valor de B | 3 |

| Valor de C | 2 |

| Valor de D | 5 |

| Valor de A | 5 |

| Primer movimiento | AD |

| Segundo movimiento | DI |

| Nodos podados | H y P |

La versión alfa-beta obtiene el mismo resultado que Minimax normal, pero no necesita evaluar H ni P. Por tanto, la poda mejora la eficiencia sin modificar la decisión final.

Analisis de los test:

Cada medición temporal se repitió 2.000 veces.

Se utilizó la mediana de los tiempos para reducir la influencia de fluctuaciones puntuales del sistema.

La medición temporal se realizó utilizando `time.perf_counter()`.

Análisis de los nodos visitados

Con profundidad 4, el árbol completo contiene:

- 6 nodos internos:

  - `A`, `B`, `C`, `D`, `G`, `J`.

- 10 hojas:

  - `E`, `F`, `H`, `K`, `L`, `I`, `M`, `N`, `O`, `P`.

Por tanto, Minimax visita:

6 + 10 = 16 nodos

La versión alfa-beta visita:

- Los mismos 6 nodos internos.

- Solo 8 hojas.

Total:

6 + 8 = 14 nodos

Nota: Estos porcentajes son independientes del ordenador utilizado.

| Métrica | Minimax | Alfa-beta | Mejora |

| Nodos internos | 6 | 6 | 0 % |

| Hojas evaluadas | 10 | 8 | 20 % |

| Nodos totales | 16 | 14 | 12,5 % |

| Valor raíz | 5 | 5 | Mismo resultado |

| Primer movimiento | `D` | `D` | Mismo resultado |

Tiempos obtenidos

Los tiempos siguientes se obtuvieron ejecutando 2.000 repeticiones por profundidad en el entorno local. Se muestra la mediana en microsegundos por movimiento.

| Profundidad | Minimax (µs) | Alfa-beta (µs) | Reducción temporal aproximada |

| 1 | 101,49 | 100,18 | 1,3 % |

| 2 | 208,39 | 180,46 | 13,4 % |

| 3 | 348,47 | 289,77 | 16,8 % |

| 4 | 355,50 | 292,80 | 17,6 % |

## 3. Documentación del diseño de la heurística

### a. Revisión de trabajos previos sobre estrategias de Reversi

El diseño de funciones de evaluación heurística para Reversi ha sido ampliamente estudiado en la literatura de Inteligencia Artificial. A diferencia de juegos como el ajedrez o las damas, Reversi presenta una dinámica no monótona: maximizar el número de piezas propias durante la apertura y el medio juego suele ser una estrategia perdedora, pues regala opciones de captura al adversario y restringe los movimientos propios.

Buro (1997, 1999) demostró con el desarrollo de Logistello el motor que derrotó al campeón mundial humano Takeshi Murakami por 6–0— que una función de evaluación competitiva debe estructurarse a través de características posicionales, control de movilidad y fases dinámicas. Específicamente, Buro formalizó la descomposición del tablero en patrones posicionales ponderados, la minimización de piezas frontera (evaporación en apertura) y el cálculo de la estabilidad de discos en bordes y esquinas.

Por su parte, Rosenbloom (1982) demostró en el sistema **IAGO** la importancia crítica de la movilidad (tanto actual como potencial) y el valor de las casillas de anclaje inalterables (esquinas), junto con la penalización severa de las casillas adyacentes $X$ y $C$ mientras la esquina permanezca libre. Estos principios, combinados con la formulación estándar de búsqueda adversarial Minimax con poda alfa-beta formalizada por Russell y Norvig (2021), constituyen la base teórica sobre la que se han estructurado nuestras heurísticas.

### b. Descripción del proceso de diseño

El diseño y ajuste de las funciones de evaluación se ejecutó de forma iterativa y empírica, guiado por los resultados de los torneos internos con tokens frente a las heurísticas de referencia del profesorado y contra el jugador aleatorio; también con los resultados de nuestro benchmark propio.

**1. Planificación inicial e hipótesis de trabajo**

Se plantearon tres variantes para cubrir distintos perfiles estratégicos sin violar la restricción de tiempo (no exceder 10 veces el tiempo base de la heurística trivial para evitar el *timeout*):

- **Hipótesis 1:** La movilidad no debe calcularse invocando funciones completas del motor en cada nodo; un escaneo de casillas vacías contiguas a fichas frontera permite derivar movimientos legales en una sola pasada $O(N)$.

- **Hipótesis 2:** En la apertura debe aplicarse una tasa de paridad negativa (estrategia de evaporación) para no comprometer casillas prematuramente.

- **Hipótesis 3:** Las casillas bloqueadas 'O' (B1, H2, A7, G8) alteran la estabilidad de los bordes; una ficha apoyada contra un obstáculo adquiere invariabilidad idéntica a la que otorga una esquina.

**2. Evaluación empírica y evolución entre torneos**
En el segundo torneo general, las versiones iniciales obtuvieron un rendimiento competitivo pero subóptimo:
Nota: los nombres de las heurísticas los hemos cambiado en cada torneo.

| Heurística (Nombre v1) | Puntuación global | Posición aprox. | Score 1 (Estándar) | Score 2 (Obstáculos) | Timeouts |
|---|---|---|---|---|---|
| FarmeoAura | 322 | ~33.º | 168 | 154 | 0 |
| JoeBidenHeuristic | 308 | ~41.º | 139 | 169 | 0 |
| EinsteinyTesla | 289 | ~48.º | 124 | 165 | 0 |

**Diagnóstico de fallos identificado:**

1. *Efecto horizonte en el endgame temprano:* JoeBidenHeuristic cortaba abruptamente la evaluación posicional en total > 52, devolviendo únicamente la diferencia de piezas. A profundidad 3 o 4, el árbol ignoraba que regalar una esquina en la jugada 53 causaba una derrota masiva en la jugada 57.
2. *Fragilidad ante obstáculos:* Los escaneos de bordes omitían los obstáculos si la esquina inicial estaba libre, perdiendo la noción de anclaje indestructible que proporcionan B1, H2, A7 y G8.
3. *Visión ciega en profundidad $d$:* En el último nivel explorado por Minimax, las jugadas que cedían esquinas al rival en el turno $d + 1$ eran puntuadas falsamente como favorables.

**3. Resultados sistemáticos en torneos internos (Tokens de prueba)**
Tras implementar el escaneo de estabilidad anclada a obstáculos (_edge_stability), la anticipación de amenazas a esquinas en $d + 1$ (_mobility_and_corner_threats) y retrasar el cierre puro de material a total >= 59, se realizaron pruebas sistemáticas con tokens:

**Rendimiento frente a heurísticas de referencia del profesorado (Torneo Interno):**

| Heurística | Tablero Estándar (vs Ref 1–5) | Tablero con Obstáculos (vs Ref 1–5) | Total vs Referencias | Efectividad global |
|---|---|---|---|---|
| Bulin47Heuristic | **10 / 10** (100%) | **10 / 10** (100%) | **20 / 20** | **100.0%** |
| IMSERSOheuristic | **10 / 10** (100%) | 9 / 10 (90%) | **19 / 20** | **95.0%** |
| JeanDelPozoHeuristic | **10 / 10** (100%) | 9 / 10 (90%) | **19 / 20** | **95.0%** |
| **Balance conjunto Pareja 08** | **30 / 30 (100%)** | **28 / 30 (93.3%)** | **58 / 60** | **96.7%** |

*Récord específico contra* heuristica_referencia_5 *(máximo nivel del profesorado):* **12 victorias en 12 partidas (100%)**.

**Rendimiento frente al jugador aleatorio (**random**) a profundidad 1 (50 repeticiones por tablero):**

- Bulin47Heuristic: 49/50 (Estándar) + 47/50 (Obstáculos) = **96 / 100 (96.0%)**.
- IMSERSOheuristic: 50/50 (Estándar) + 48/50 (Obstáculos) = **98 / 100 (98.0%)**.
- JeanDelPozoHeuristic: 50/50 (Estándar) + 49/50 (Obstáculos) = **99 / 100 (99.0%)**.

### c. Descripción de la heurística enviada finalmente

La entrega final contiene tres variantes adaptativas implementadas sobre la clase abstracta StudentHeuristic: Bulin47Heuristic, IMSERSOheuristic y JeanDelPozoHeuristic. Todas calculan estrictamente su evaluación desde la perspectiva del jugador raíz (state.player_max.label), evitando distorsiones de polaridad en niveles impares del árbol de Minimax.

**1. Componentes del estado evaluados**

La función combina linealmente siete rasgos ortogonales del juego:

1. **Puntuación Posicional Dinámica ($P$):** Ponderación matricial de casillas en $8 \times 8$. Se penalizan drásticamente las casillas $X$ $(-75)$ y $C$ $(-28)$ mientras la esquina esté vacía. Si la esquina ha sido capturada por el jugador evaluado, dichas casillas pasan a ser anclajes seguros y su peso se conmuta dinámicamente a $+18$.

2. **Control de Esquinas ($C$):** Balance directo entre esquinas propias y del rival ($C_{max} - C_{min}$) con peso preponderante ($+94$ a $+95$).

3. **Estabilidad de Bordes y Obstáculos ($S$):** Recorrido de los 4 bordes mediante `_edge_stability`. Las piezas contiguas a una esquina propia o protegidas por un obstáculo `'O'` no pueden ser volteadas en flanqueos de borde y se puntúan como fichas estables ($+29.5$ a $+30.0$).

4. **Movilidad Relativa ($M$):** Proporción normalizada de acciones legales:

$$M = 100 \times \frac{Moves_{max} - Moves_{min}}{Moves_{max} + Moves_{min}}$$

5. **Bonus por Ahogo (*Wipeout* $W$):** Si $Moves_{min} = 0$ y $Moves_{max} > 0$, se añade un incentivo crítico $(+75)$ por forzar al oponente a pasar turno.

6. **Amenaza a Esquinas en Horizonte ($T$):** Detección en un paso $(d + 1)$ de si alguno de los movimientos legales calculados corresponde a una esquina libre, ponderando con mayor urgencia si el turno en el nodo actual le corresponde al jugador evaluado o al rival.

7. **Penalización de Frontera ($F$):** Diferencial de fichas en contacto con casillas vacías ($-8.5 \times (F_{max} - F_{min})$) para forzar configuraciones cerradas y minimizar apoyos de captura al contrario.

8. **Paridad Material Progresiva ($\Pi$):** Diferencia porcentual de fichas en función de la etapa de la partida:

$$\Pi = 100 \times \frac{Coins_{max} - Coins_{min}}{Coins_{max} + Coins_{min}}$$

**2. Formulación matemática de la heurística insignia (`Bulin47Heuristic`)**
Para un estado no terminal con $N = Coins_{max} + Coins_{min}$ piezas ocupadas:

- **Si $N \geq 59$ (Final absoluto):**

  - $V(s) = 150.0 \times (Coins_{max} - Coins_{min}) + P$

- La diferencia material domina de forma absoluta, manteniendo el componente posicional únicamente como criterio de desempate determinista.

- **Si $N < 59$ (Apertura y Medio Juego):**

- $V(s) = 1.15 \cdot P + 1.30 \cdot M + W + 95.0 \cdot \Delta C + \Delta T + 30.0 \cdot \Delta S + 8.0 \cdot \Delta E + w_{\pi}(N) \cdot \Pi - 8.5 \cdot \Delta F -$

### d. Otra información relevante

1. **Eficiencia algorítmica y prevención de *timeouts*:**
   El enunciado establece una penalización estricta por descalificación si el jugador excede 10 veces el tiempo de la heurística base. Para garantizar la cota temporal requerida, se eliminaron todas las llamadas a funciones de copia profunda y a métodos de búsqueda de movimientos del motor. `_board_features` calcula las piezas, fronteras, casillas peligrosas y el conjunto de casillas vacías contiguas en una única iteración de orden $O(N)$ sobre el diccionario del tablero ($N \leq 60$). La movilidad exacta posterior (`_legal_move_count_fast`) restringe su exploración exclusivamente a dichas casillas candidatas, ejecutando cada evaluación en microsegundos y registrando **0 timeouts en 360 partidas de torneo**.
2. **Adaptabilidad a tableros con obstáculos:**
   En la variante con celdas bloqueadas en `B1`, `H2`, `A7` y `G8`, las reglas impiden colocar fichas y bloquean líneas de captura. Nuestro algoritmo procesa las coordenadas `'O'` como muros inalterables dentro de `_edge_stability`, lo que permitió a `Bulin47Heuristic` maximizar la ocupación de las casillas `C1`, `H3`, `A6` y `F8`, logrando un **pleno perfecto de 10/10 en el escenario de obstáculos**.

3. **Complementariedad del conjunto de 3 heurísticas entregadas:**

   - `Bulin47Heuristic` actúa como perfil de máxima solidez geométrica y estabilidad en bordes (100% de victorias frente a referencias).
   - `IMSERSOheuristic` equilibra una movilidad sostenida con mayor ponderación de ahogo al oponente (95% de victorias).
   - `JeanDelPozoHeuristic` acentúa la evaporación en la apertura ($w_{\pi} = -0.35$), castigando con especial eficacia a heurísticas rivales que priorizan la captura temprana de fichas.

## 5. Pruebas y reproducibilidad

Las pruebas unitarias verifican seis casos y se ejecutan con los comandos especificados en el README

El benchmark de heurísticas se ejecuta a profundidad 4, que es la máxima indicada en el enunciado. Para una comprobación rápida se pueden limitar partidas y tiempo por movimiento; el README explica las opciones y los tableros oficiales, incluido el tablero con obstáculos B1, H2, A7 y G8.

## 6. Bibliografía

Buro, M. (1997). The Othello match of the year: An account of the 1997 computer Othello match. ICCA Journal, 20(3), 3–14.

Russell, S. J., & Norvig, P. (2021). Artificial intelligence: A modern approach (4th ed.). Pearson.

Buro, M. (1995). Probabilistic quiescence search for computer Othello. AAAI-95 Proceedings, 366–370.